Machine Coding: Signup Form with Validation

- **Problem:** Build a signup form (username, email, password, confirm password) with real-time validation, duplicate-email check, show/hide password, and a list of registered users that can be deleted and survives refresh.
- **Tech stack:** React 19 + Vite + Tailwind CSS (JavaScript)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Assumptions written down
Use cases + architecture + state	8 min	Table, tree, state shape
Core build (form, validation, submit)	25 min	Working form
States, edge cases, a11y, persistence	12 min	Polished behaviour
Test manually + wrap-up	10 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"I'll derive errors from values instead of storing them, so they can never go stale."* If you run out of time, a clear plan you talked through still scores well.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Fields: username, email, password, confirm password.
- Validate every field with clear messages: required, format, password strength, confirm matches password.
- Show errors at the right time: after the user leaves a field (blur), and for every field on submit.
- Show/hide password toggle.
- Submit to an (async) API; prevent duplicate emails; show success or error.
- List registered users; delete a user; keep the list after a refresh (localStorage).

Non-functional requirements

Concern	What it means here
Performance	4 fields, so re-validating everything on each keystroke is cheap. No memoization needed.
Scalability	Field config array + pure rule functions, so a new field is one config entry and one rule.
Error handling	Field errors, server field errors (duplicate email), general server error, corrupted localStorage.
Accessibility	`label` / `id` pairs, `aria-invalid`, `aria-describedby`, `role="alert"`, focus the first invalid field.
Reusability	Validators are pure functions shared by the form (UX) and the mock server (trust).
Extensibility	Swapping the mock API for `fetch` changes one file only.

Clarifying questions to ask the interviewer

1. What are the exact rules for username and password (length, allowed characters)?
2. When should errors appear: while typing, on blur, or only on submit?
3. Is there a real API, or should I mock it? Who decides "email already exists"?
4. Should the registered list persist across refresh?
5. Should the submit button be disabled while the form is invalid?
6. Any accessibility or design-system constraints?

Assumptions we will build with

- Username: 3-15 characters, letters/numbers/underscore. Password: at least 8 characters with upper, lower, number and special character.
- Errors show **after blur** (not on the first keystroke, which feels hostile), and on submit for all fields.
- The submit button stays **enabled** so pressing it reveals what's wrong; it is disabled only while submitting.
- The API is mocked with a promise + localStorage; the mock re-validates (never trust the client).
- Email is trimmed and lowercased before it is sent, so `A@B.com` and `a@b.com` count as the same email.
- Passwords are **never** stored in localStorage.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Type in a field the user hasn't left yet	Value updates; no error shown yet
Leave (blur) a field with an invalid value	Error appears under it; input gets a red border
Fix an invalid value	Error disappears while typing (field is already touched)
Change password after confirm already matched	Confirm shows "Passwords do not match" immediately
Click Sign up with empty/invalid fields	All errors shown; focus moves to the first invalid field; no API call
Click Sign up with valid fields	Button shows "Signing up..." and is disabled; inputs read-only
Press Enter repeatedly while submitting	Only one request is sent
API succeeds	Success message, form reset, user added to the list
API: email already registered	Error under the email field, focus moves to email
Edit email after the duplicate error	Server error clears
API fails (network)	General error message above the button; values kept
Toggle "Show passwords"	Both password fields switch between hidden and plain text
Click Delete on a user	User removed from the list and from storage
Refresh the page	Registered users are still there
No users	"No users registered yet."

3. Design the architecture

Module	Responsibility
`App`	Root; renders the feature
`SignupForm`	Owns form state, renders fields, handles change/blur/submit, renders the users list
`utils/validators.js`	Pure rules: `validateField(name, values)`, `validateAll(values)`
`api/mockApi.js`	Fake backend: `registerUser`, `getUsers`, `deleteUser` (localStorage + delay)

```
App
└── SignupForm  ──uses──▶ validators.js  (pure rules)
     │          ──calls─▶ mockApi.js ──uses──▶ validators.js (server re-check)
     ├── form: FIELDS.map → label + input + error
     ├── show-password checkbox, status message, submit button
     └── users list: li + Delete button
```

Start simple. One component is fine for 4 fields. Split out a `FormField` component or a `useForm` hook only when a second form needs them.

4. Design the state / data model

State	Purpose
`values`	Current input values `{ username, email, password, confirmPassword }`
`touched`	Which fields the user has left: controls *when* errors show
`status`	`idle | submitting | success | error` (one enum, not four booleans)
`serverError`	`{ message, field? }` from the API (field-level or general)
`showPassword`	Toggle for the password inputs
`users`	Registered users list (lazy-initialised from storage)
`lastUser`	Name for the success message
`inputRefs` (ref)	DOM nodes, so we can focus the first invalid field

```
const [values, setValues] = useState(INITIAL_VALUES);
const [touched, setTouched] = useState({});
const [status, setStatus] = useState("idle"); // idle | submitting | success | error
const [serverError, setServerError] = useState(null); // { message, field? }
const [showPassword, setShowPassword] = useState(false);
const [users, setUsers] = useState(getUsers); // lazy init: reads storage once
const inputRefs = useRef({});

const errors = validateAll(values);          // derived
const isSubmitting = status === "submitting"; // derived
```

Senior note

- **Derived, not stored:** `errors`, `isSubmitting`, the visible error per field, and `users.length`. Storing `errors` in state (as the first version did) means you must remember to re-validate confirm whenever password changes. Deriving it makes that bug impossible.
- **Ownership:** all form state lives in `SignupForm` (the only consumer). The `users` list is owned by the "server" (`mockApi`); the component keeps a copy and refreshes it from each API result.

5. Design the data structures and contracts

Field config (drives rendering, so a new field is one line):

```
{ name: "email", label: "Email", type: "email", autoComplete: "email" }
```

Stored user (stable `id` from `crypto.randomUUID()` so React keys and delete don't depend on the array index; no password):

```
{ "id": "6f1c…", "username": "ashu_21", "email": "ashu@test.com", "createdAt": "2026-11-
01T10:00:00.000Z" }
```

API contracts (mocked)

Call	Input	Success	Failure
`registerUser` (POST /api/users)	`{ username, email, password }`	`user` object (above)	`Error{ message, field? }` e.g. `{ message: "This email is already registered", field: "email" }`; `{ message: "Server unavailable..." }` (no field)
`getUsers` (GET /api/users)	-	`user[]` (empty on corrupt storage)	-
`deleteUser` (DELETE /api/users/:id)	`id`	remaining `user[]`	-

To demo the network-error state, use any email ending in `@fail.test`.

6. Trace the data flow

Submit with valid values:

```
User presses Enter / clicks Sign up
  ↓ handleSubmit: preventDefault, ignore if already submitting
  ↓ setTouched(all) → every error becomes visible
  ↓ errors = validateAll(values) → no invalid field
  ↓ setStatus("submitting") → button "Signing up...", inputs read-only
  ↓ registerUser({ username.trim(), email.trim().toLowerCase(), password })
  ↓   mock: wait 800ms → re-validate → check duplicate → save user (no password) → return user
  ↓ setUsers([...prev, user]), reset values + touched, setStatus("success")
  ↓ UI: success message, empty form, new row in "Registered users"
(on rejection: setServerError → error under the field or above the button, focus that field)
```

7. Important technical challenges

Challenge 1: cross-field validation goes stale

- *Problem:* Password `Abc@1234`, confirm `Abc@1234` (valid). User changes password to `Abc@12345`. If errors live in state and are only recomputed for the edited field, confirm still says "valid".
- *Solution:* Validators take **all values**, and errors are derived every render.

```
case "confirmPassword":
  if (!value) return "Please confirm your password";
  return value === values.password ? "" : "Passwords do not match";
// in the component:
const errors = validateAll(values);
```

Challenge 2: when to show errors (UX)

- *Problem:* Showing "Invalid email" after the first keystroke `a` is noisy; showing errors only on submit is too late.
- *Solution:* A `touched` map. An error shows once the field is blurred, or on submit (which marks every field as touched). Once touched, the error updates live as the user fixes it.

```
const errorFor = (name) => {
  if (!touched[name]) return "";
  if (errors[name]) return errors[name];
  return serverError?.field === name ? serverError.message : "";
};
```

Challenge 3: async submit, double submit and server errors

- *Problem:* The user hits Enter 3 times and 3 requests go out. Or the server rejects a duplicate email, but the client can't know that ahead of time.
- *Solution:* One `status` enum; ignore submit while submitting; map a server error with a `field` back onto that input and focus it; clear it when the field changes.

```
if (isSubmitting) return;
...
} catch (err) {
  setServerError({ message: err.message, field: err.field });
  setStatus("error");
  if (err.field) inputRefs.current[err.field]?.focus();
}
```

Challenge 4: accessibility and focus management

- *Problem:* A screen-reader user submits and hears nothing. The first version used a placeholder instead of a label and had no `id` matching `htmlFor`.
- *Solution:* Pair each label with an input `id`, link the error with `aria-describedby`, mark it `aria-invalid`, announce it with `role="alert"`, and focus the first invalid field. Use `readOnly` (not `disabled`) while submitting, because a disabled input can't receive focus when the error comes back.

```
<input id={field.name} aria-invalid={Boolean(error)}
  aria-describedby={error ? `${field.name}-error` : undefined} readOnly={isSubmitting} />
{error && <p id={`${field.name}-error`} role="alert">{error}</p>}
```

> Bugs fixed from the earlier `AdvancedValidation.jsx`: `value={formData[field]}` (an object key, so inputs were uncontrolled), confirm not re-checked when password changes, submit sent data without validating, the show/hide `<button>` inside the form submitted it (needs `type="button"`), and the list keyed by array index.

8. Implement in the correct order

#	Step	Working result
1	Field config + controlled inputs with labels	Typing works
2	`validators.js` with pure rules + node self-check	Rules proven without UI
3	Derive `errors`; show under each field	Errors visible
4	`touched` + `onBlur`	Errors appear at the right time
5	Submit: mark all touched, block if invalid, focus first invalid	Safe submit
6	Mock API with delay + `status` enum	Loading state
7	Success path: reset form, success message	Happy path done
8	Server errors: duplicate email → field error; network → general	Error paths done
9	Users list from storage + delete + empty state	Persistence done
10	Show/hide password, a11y attributes, double-submit guard	Polished

9. Cover all states and edge cases

State / edge case	Handling
Initial	Empty fields, no errors, "No users registered yet." if storage is empty
Typing (untouched)	No error shown
Touched + invalid	Red border + message, `aria-invalid`
Submitting	Button disabled with "Signing up...", inputs read-only, Enter ignored
Success	Green `role="status"` message, form reset, user appended
Empty list	Friendly empty message
Server field error	Shown under the field, focused, cleared on edit
Server general error	Shown above the button, values kept for retry
Whitespace-only username	"Username is required" (trimmed)
`A@B.com` vs `a@b.com`	Normalised, so it's treated as a duplicate
Password changed after confirm	Confirm error appears immediately
Corrupted / blocked localStorage	`getUsers` returns `[]` instead of crashing
Storage quota full on save	`registerUser` rejects, so the general error shows
Browser autofill	Works through `onChange` + `autoComplete` hints

10. Final structure

```
_01_form_validation/
├── index.html
└── src/
    ├── main.jsx
    ├── App.jsx                ← renders <SignupForm />
    ├── index.css              ← @import "tailwindcss"
    ├── api/
    │   └── mockApi.js         ← fake backend (localStorage + delay)
    ├── components/
    │   ├── SignupForm.jsx     ← the solution
    │   └── AdvancedValidation.jsx  ← earlier attempt, kept for comparison
    └── utils/
        ├── validators.js      ← pure rules
        └── validators.check.js← `node src/utils/validators.check.js`
```

11. Full working code

`src/utils/validators.js`

```
// Pure validation rules: same functions run in the form (UX) and in the mock server (trust).
const PATTERNS = {
  username: /^[a-zA-Z0-9_]{3,15}$/,
  email: /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/,
  password: /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z\d]).{8,}$/,
};

// Returns "" when valid, otherwise the message to show. Takes all values for cross-field rules.
export function validateField(name, values) {
  const value = values[name] ?? "";
  switch (name) {
    case "username":
      if (!value.trim()) return "Username is required";
      return PATTERNS.username.test(value.trim()) ? "" : "3-15 characters: letters, numbers or
underscore";
    case "email":
      if (!value.trim()) return "Email is required";
      return PATTERNS.email.test(value.trim()) ? "" : "Enter a valid email, e.g. name@example.com";
    case "password":
      if (!value) return "Password is required";
      return PATTERNS.password.test(value) ? "" : "Min 8 characters with upper, lower, number and
special character";
    case "confirmPassword":
      if (!value) return "Please confirm your password";
      return value === values.password ? "" : "Passwords do not match";
```

```
    default:
      return "";
  }
}

export function validateAll(values) {
  return Object.fromEntries(Object.keys(values).map((name) => [name, validateField(name, values)]));
}
```

src/api/mockApi.js

```
import { validateAll } from "../utils/validators";

// shortcut: localStorage stands in for a backend; swap these functions for fetch() calls when a real
API exists.
const KEY = "registeredUsers";
const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
const fail = (message, field) => Object.assign(new Error(message), { field });

export function getUsers() {
  try {
    const data = JSON.parse(localStorage.getItem(KEY));
    return Array.isArray(data) ? data : [];
  } catch {
    return []; // corrupted or blocked storage: start empty instead of crashing
  }
}

function saveUsers(users) {
  localStorage.setItem(KEY, JSON.stringify(users));
}

// POST /api/users  { username, email, password } -> 201 { id, username, email, createdAt }
export async function registerUser({ username, email, password }) {
  await delay(800);
  if (email.endsWith("@fail.test")) throw fail("Server unavailable. Please try again.");

  // Never trust the client: the server re-runs the same rules.
  const errors = validateAll({ username, email, password, confirmPassword: password });
  const field = Object.keys(errors).find((name) => errors[name]);
  if (field) throw fail(errors[field], field);

  const users = getUsers();
  if (users.some((user) => user.email === email)) throw fail("This email is already registered",
"email");

  // The password is never stored: a real server would hash it.
```

```
  const user = { id: crypto.randomUUID(), username, email, createdAt: new Date().toISOString() };
  saveUsers([...users, user]);
  return user;
}

// DELETE /api/users/:id -> remaining users
export function deleteUser(id) {
  const next = getUsers().filter((user) => user.id !== id);
  saveUsers(next);
  return next;
}
```

src/components/SignupForm.jsx

```
import { useRef, useState } from "react";
import { validateAll } from "../utils/validators";
import { deleteUser, getUsers, registerUser } from "../api/mockApi";

const FIELDS = [
  { name: "username", label: "Username", type: "text", autoComplete: "username" },
  { name: "email", label: "Email", type: "email", autoComplete: "email" },
  { name: "password", label: "Password", type: "password", autoComplete: "new-password" },
  { name: "confirmPassword", label: "Confirm password", type: "password", autoComplete: "new-password"
},
];
const INITIAL_VALUES = { username: "", email: "", password: "", confirmPassword: "" };
const ALL_TOUCHED = Object.fromEntries(FIELDS.map((f) => [f.name, true]));

const SignupForm = () => {
  const [values, setValues] = useState(INITIAL_VALUES);
  const [touched, setTouched] = useState({});
  const [status, setStatus] = useState("idle"); // idle | submitting | success | error
  const [serverError, setServerError] = useState(null); // { message, field? }
  const [showPassword, setShowPassword] = useState(false);
  const [users, setUsers] = useState(getUsers);
  const [lastUser, setLastUser] = useState("");
  const inputRefs = useRef({});

  // Derived, not stored: errors can never go stale (e.g. confirm after password changes).
  const errors = validateAll(values);
  const isSubmitting = status === "submitting";

  const errorFor = (name) => {
    if (!touched[name]) return "";
    if (errors[name]) return errors[name];
    return serverError?.field === name ? serverError.message : "";
  };
```

```
const handleChange = (e) => {
  const { name, value } = e.target;
  setValues((prev) => ({ ...prev, [name]: value }));
  if (serverError?.field === name) setServerError(null);
  if (status === "success") setStatus("idle");
};

const handleBlur = (e) => {
  const { name } = e.target;
  setTouched((prev) => ({ ...prev, [name]: true }));
};

const handleSubmit = async (e) => {
  e.preventDefault();
  if (isSubmitting) return; // guard against double submit (Enter key spam)

  setTouched(ALL_TOUCHED);
  const firstInvalid = FIELDS.find((f) => errors[f.name]);
  if (firstInvalid) {
    inputRefs.current[firstInvalid.name]?.focus();
    return;
  }

  setStatus("submitting");
  setServerError(null);
  try {
    // confirmPassword is a UI-only field, it is not sent.
    const user = await registerUser({
      username: values.username.trim(),
      email: values.email.trim().toLowerCase(),
      password: values.password,
    });
    setUsers((prev) => [...prev, user]);
    setLastUser(user.username);
    setValues(INITIAL_VALUES);
    setTouched({});
    setShowPassword(false);
    setStatus("success");
  } catch (err) {
    setServerError({ message: err.message, field: err.field });
    setStatus("error");
    if (err.field) inputRefs.current[err.field]?.focus();
  }
};

const handleDelete = (id) => setUsers(deleteUser(id));
```

```
  return (
    <div className="p-10">
      <h1 className="text-center font-bold text-2xl">Signup form validation</h1>

      <div className="mt-10 flex flex-col items-center gap-8">
        <form className="w-full max-w-[500px] border p-6 rounded" onSubmit={handleSubmit} noValidate>
          {FIELDS.map((field) => {
            const error = errorFor(field.name);
            const isPassword = field.type === "password";
            return (
              <div key={field.name} className="mb-4">
                <label htmlFor={field.name} className="font-medium">
                  {field.label}
                </label>
                <input
                  ref={(el) => {
                    inputRefs.current[field.name] = el;
                  }}
                  id={field.name}
                  name={field.name}
                  type={isPassword && showPassword ? "text" : field.type}
                  autoComplete={field.autoComplete}
                  value={values[field.name]}
                  onChange={handleChange}
                  onBlur={handleBlur}
                  readOnly={isSubmitting}
                  aria-invalid={Boolean(error)}
                  aria-describedby={error ? `${field.name}-error` : undefined}
                  className={`border w-full px-2 py-1 mt-2 rounded ${error ? "border-red-500" : ""}`}
                />
                {error && (
                  <p id={`${field.name}-error`} role="alert" className="text-red-500 text-sm mt-1">
                    {error}
                  </p>
                )}
              </div>
            );
          })}

          <label className="flex items-center gap-2 text-sm">
            <input type="checkbox" checked={showPassword} onChange={(e) =>
setShowPassword(e.target.checked)} />
            Show passwords
          </label>

          {status === "error" && !serverError?.field && (
```

```
            <p role="alert" className="mt-4 text-red-600">
              {serverError?.message}
            </p>
          )}
          {status === "success" && (
            <p role="status" className="mt-4 text-green-700">
              Account created for {lastUser}.
            </p>
          )}

          <button
            type="submit"
            disabled={isSubmitting}
            className="mt-6 w-full bg-blue-500 p-2 cursor-pointer text-white hover:bg-blue-700
disabled:cursor-not-allowed disabled:opacity-60"
          >
            {isSubmitting ? "Signing up..." : "Sign up"}
          </button>
        </form>

        <section className="w-full max-w-[500px]">
          <h2 className="font-bold text-xl">Registered users ({users.length})</h2>
          {users.length === 0 ? (
            <p className="mt-2 text-gray-500">No users registered yet.</p>
          ) : (
            <ul className="mt-2 divide-y border rounded">
              {users.map((user) => (
                <li key={user.id} className="flex items-center justify-between p-2">
                  <span>
                    {user.username} <span className="text-gray-500">({user.email})</span>
                  </span>
                  <button
                    type="button"
                    onClick={() => handleDelete(user.id)}
                    aria-label={`Delete ${user.email}`}
                    className="text-red-600 hover:underline"
                  >
                    Delete
                  </button>
                </li>
              ))}
            </ul>
          )}
        </section>
      </div>
    </div>
  );
```

```
};

export default SignupForm;
```

`src/App.jsx`

```
import SignupForm from "./components/SignupForm";

const App = () => <SignupForm />;

export default App;
```

12. Wrap-up

Edge cases handled: required vs format errors, trimmed input, case-insensitive duplicate email, stale confirm-password, double submit, server field vs general errors, corrupted storage, empty list, focus management, no password persisted.

Trade-offs to mention to the interviewer

- *Validating all fields on every keystroke* is simple and cheap for 4 fields; for 50+ fields or expensive rules, validate only the changed field plus its dependents.
- *Hand-written validators vs Zod/Yup + React Hook Form:* no dependencies and easy to explain in an interview; in production a schema library gives types and less code per form.
- *Button enabled while invalid:* pressing it reveals the errors (better discoverability) instead of leaving a dead button the user can't understand.
- *localStorage as the backend:* fine for a demo; real duplicate checks must happen on the server (a race between two tabs can't be solved on the client).
- *One component:* fastest to build; extract `FormField` / `useForm` once a second form exists.

What to add with more time

- Tests: React Testing Library for the use-case table (blur → error, submit → focus, duplicate → email error), plus the existing validator self-check moved to Vitest.
- An async "username available?" check on blur, debounced, which ignores stale responses.
- A password strength meter and a rule checklist.
- Abort the request on unmount (`AbortController`) when using a real `fetch` .
- `useReducer` or a `useForm` hook when the form grows; i18n for the messages.

13. Counter questions the interviewer may ask

Requirements and design

Q1. Why validate on the client at all if the server validates anyway?

Client validation is for UX: instant feedback with no network round trip. Server validation is for security and data integrity, because the client can always be bypassed (DevTools, Postman, disabled JS). You need both; the mock server here re-runs the same rules to show that.

Q2. When should errors appear: on change, on blur, or on submit?

On blur first (don't shout at the first keystroke), then live on change once the field is touched, and for every field on submit. That's the `touched` map. Errors-only-on-submit is too late; errors-on-first-key feels hostile.

Q3. Why keep the submit button enabled when the form is invalid?

A disabled button gives no reason why. An enabled one reveals all errors and focuses the first invalid field. Disable it only while submitting, to stop double submits.

Q4. Why `noValidate` on the form? Isn't HTML5 validation good enough?

Native bubbles can't be styled consistently, the messages vary by browser, and cross-field rules (confirm password) aren't possible. We keep `type="email"` and `autoComplete` for keyboards and autofill, but show our own accessible messages.

State design

Q5. Why derive `errors` instead of storing them in state?

Stored errors go stale. Change the password and the old "confirm matches" result stays wrong unless you remember to re-validate confirm too. `validateAll(values)` on every render can never be out of sync, and with 4 fields it costs nothing.

Q6. Why one `status` enum instead of `isLoading`, `isError`, `isSuccess` booleans?

Booleans allow impossible combinations (`isLoading && isSuccess`). One enum (`idle | submitting | success | error`) makes exactly one state true at a time.

Q7. Why is `confirmPassword` not sent to the API?

It's a UI-only check to catch typos. The server only needs the password; sending both just adds noise (and one more copy of a secret in logs).

Q8. Where would the form state live in a big app?

Still local: form state rarely needs to be global. For many forms, use React Hook Form (uncontrolled inputs, fewer re-renders) with a Zod schema shared with the backend.

Hard parts

Q9. How does the confirm-password error update when only the password changes?

Validators receive all values (`validateField(name, values)`), and errors are recomputed from the whole `values` object on every render. Editing the password re-renders, so confirm is checked against the new password automatically.

Q10. The server says "email already registered". How do you show it?

The API rejects with `{ message, field: "email" }` . We store it as `serverError` , show it under that field, focus it, and clear it as soon as the user edits that field. Errors without a `field` show above the button.

Q11. Why `readOnly` instead of `disabled` on inputs while submitting?

A disabled input can't receive focus. When the server error comes back we need to focus the email field, and that works only if it's read-only. Disabled inputs are also skipped by screen readers and not submitted with native forms.

Q12. How do you prevent double submission?

Return early if `status === "submitting"`, and disable the button while submitting. With a real API, also use an idempotency key, so the server ignores a duplicate request.

Implementation steps

Q13. Write a regex for "min 8 characters, with upper, lower, number and special character".

`/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z\d]).{8,}$/`. Each `(?=…)` lookahead checks one rule without consuming characters; `.{8,}` checks the length. Mention that a strength meter / rule checklist is friendlier than one long message.

Q14. Why trim the username and lowercase the email before sending?

" Ashu" and "ashu" would otherwise be different users, and "A@B.com" vs "a@b.com" would bypass the duplicate check. Normalise at the boundary, once.

Q15. How would you test this form?

Unit-test the pure validators (done with a node assert script). Use React Testing Library for: blur → error, submit empty → focus on the first field, valid submit → success message, duplicate email → error under email, with a mocked API.

Performance and scale

Q16. The form grows to 50 fields. What changes?

Validating every field on each keystroke may become noticeable, and every keystroke re-renders the whole form. Options: validate only the changed field + its dependents, split into memoised field components, or use React Hook Form (uncontrolled inputs). Also consider a multi-step form.

Q17. How would you check "username available" while typing?

An async validator on blur (or debounced on change, ~400 ms), with `AbortController` to cancel stale checks, a "checking..." state, and the server still re-checks on submit (two users can race for the same name).

Trade-offs and "why not X?"

Q18. Why not Formik / React Hook Form / Zod?

In production I'd use React Hook Form + Zod: less code, a schema shared with the backend, typed errors. In an interview, hand-writing it shows you understand controlled inputs, derived state and validation timing. Then mention the libraries.

Q19. Why not store users with passwords in localStorage?

Never store passwords on the client: anything in localStorage is readable by any script on the page (XSS). Here localStorage only mocks the server, and the password is dropped; a real server hashes it (bcrypt/argon2).

Q20. What security issues does a signup form have beyond validation?

HTTPS only; rate limiting and a CAPTCHA against bots; never tell the user *which* field was wrong on login (avoids account enumeration); CSRF protection if cookies are used; escape all user input when it's rendered (React does this by default).